LLD Phase 5 — Starter Kit

The method, the toolbox and the tracker for the 24 practice problems | Read this once before PDF 1, then keep it open as a reference

What this kit is for. The six problem PDFs are built so you can work through Phase 5 alone, at your own pace. This kit holds everything they assume: a method that works on any problem, the design toolbox (requirements, entities, UML, layering, concurrency, failure rules), the pattern cheat sheet, how the machine-coding tests work, and a tracker for all 24 problems.

Contents

1. How the six PDFs work · the loop for each problem · the re-do schedule

2. The method for any LLD problem · time budgets for a 45-minute and a 90-minute round

3. Requirements: functional, non-functional, out of scope · the questions to ask

4. Entities and relationships · class, enum, record or field? · who owns the behaviour

5. UML class diagrams on one page

6. Layering: driver, service, repository, entity

7. Concurrency toolbox · check-then-act · deadlock and lock ordering · many servers

8. Failure rules

9. Pattern cheat sheet: all 21 patterns, the "no pattern" cases, the look-alikes

10. The machine-coding format: commands in, exact output out

11. Interview behaviour: thinking aloud, defending, "I don't know"

12. Master tracker for all 24 problems

1. How the six PDFs work

Each PDF has four problems in the same mix: **one small warm-up, two core problems, one heavy problem**. The PDFs get harder from 1 to 6, and each one mixes problem types, so you have to *choose* patterns rather than guess them from the chapter title.

PDF	Small	Core	Core	Heavy
1	Tic-Tac-Toe	Vending Machine	Library Management	**Parking Lot**
2	Deck of Cards	ATM	Splitwise	**Elevator System**
3	LRU Cache	Rate Limiter	Digital Wallet	**BookMyShow**
4	Snake and Ladder	Cache with TTL and eviction	Meeting Room Scheduler	**Chess**
5	Logging Framework	Hotel Booking	Task Scheduler	**Food Delivery**
6	Traffic Signal / Amazon Locker	Notification System	Inventory Management	**Ride-sharing**

The loop for every problem

#	Step	How
1	Read and time-box	Read the requirements once. Set a timer: the time box is printed with each problem.
2	Part A on paper	Entities, relationships, a class diagram, key methods, patterns with reasons, failure and concurrency decisions. Say your reasoning aloud as if to an interviewer.
3	Hints, only if stuck	Stuck for 15 minutes? Read **one** hint level, then go back to work. Three levels: a nudge, the shape, the key decision.
4	Part B in your IDE	Write the code and run it on the given input. Compare with the expected output (section 10). Fix until it matches exactly.
5	Part C aloud	Answer the pressure questions out loud, or record yourself. Open "defend it" answers with a concession.
6	Only now, the solution	Compare your design, not just your output. Score yourself with the 10-point checklist.
7	Log and schedule	Write every mistake in the mistake log at the back of the PDF. Put the re-do dates in your calendar.

The re-do schedule: this is what makes it stick. Re-reading a solution feels like learning and isn't. Re-designing from memory is. After each problem: **day 3**, sketch Part A again from memory (15 minutes); **day 10**, do Part A again and code the core class; **day 30**, do the whole problem under the time box with nothing open. Each PDF also opens with warm-ups that re-test problems from the earlier PDFs.

2. The method for any LLD problem

Every problem in the six PDFs, and every interview problem you'll meet, can be worked in the same seven steps. The steps are always the same; only the answers change.

#	Step	What you produce	The question to ask yourself
1	**Clarify**	A short list: functional, non-functional, out of scope	"What must it do, how well, and what am I *not* building?"
2	**Entities**	The nouns that become classes, enums and records	"What are the things, and how do they relate?"
3	**Behaviour**	The key operations and which class owns each	"Who knows the data this needs? That class owns it."
4	**Variation**	Patterns where something varies, and "nothing" where nothing does	"What changes, or comes in several kinds?"
5	**Failure**	Exceptions, clean-up, the order of steps	"If this step fails, is the operation still valid?"
6	**Concurrency**	Which shared state needs protecting, and how	"Can two requests read-decide-write the same thing at once?"
7	**Code and extend**	Core classes, a driver, then the follow-up questions	"Does my design survive the next requirement?"

Time budgets

Phase	45-minute discussion round	90-minute machine-coding round
Clarify requirements	5 min	5–10 min
Entities and class diagram	10 min	10 min (rough, on paper)
Patterns, key methods, concurrency	10 min	5 min
Code	15 min (the core class or two)	55–60 min (all of it, runnable)
Test, follow-ups, clean-up	5 min	10 min

> **The most common failure:** 30 minutes on requirements and no time left to code. Watch the clock at each phase change. If you're over, say so and move on: "I'll assume X so we can get to the design."

3. Requirements

Kind	What it is	Parking Lot examples
Functional	What the system must **do**: the features	Park a vehicle, issue a ticket, charge at exit, show free spots
Non-functional	How **well** it must do it: speed, scale, safety, concurrency, extensibility	Two gates must never get the same spot; new vehicle types must be easy to add
Out of scope	What you agree **not** to build	Online pre-booking, monthly passes, payment gateway details

Questions worth asking (pick the ones that matter)

1. Who are the users or actors? (customer, operator, admin, another system)
2. What are the 3–5 core operations? Which one matters most?
3. What kinds or types are there, and will new ones be added? (vehicle types, payment methods)
4. What are the rules and limits? (max books per member, opening hours, price formulas)
5. What happens when something fails? (payment fails, item sold out, nobody approves)
6. Can two people act on the same thing at once? (last seat, last spot, same account)
7. One machine or many servers? This decides `synchronized` versus database locking.
8. What must be remembered? History, receipts, audit logs?
9. Who needs to be told when something happens? (notifications)
10. What can I leave out for now?

In the PDFs the requirements are written out for you, so you can't ask. Each solution lists the questions a strong candidate would have asked, so you can check whether you would have thought of them.

4. Entities and relationships

The noun-and-verb hunt. Underline the nouns in the requirements: they are candidate classes. Underline the verbs: they are candidate methods. Then cut. Not every noun deserves a class.

If the noun is…	Make it	Example
A thing with an identity, data **and** behaviour that changes over time	**class**	`Spot`, `Member`, `Game`
A fixed set of kinds that differ only in **data**	**enum with fields**	`VehicleType(rate)`, `MembershipType(maxBooks, loanDays)`
A fixed set of kinds that differ in **behaviour**	**interface + classes** (Strategy, State)	`AllocationPolicy`, `MachineState`
A small bundle of values that never changes	**record**	`Ticket`, `Product`, `Player`
A single value describing something else	**field**	a spot's `id`, a book's `title`
A status that only one or two methods check	**enum field**, no State pattern	`CopyStatus`, `GameStatus`

Two rules from the patterns course. Differ by data → one class with fields. Differ by behaviour → separate classes. And: **who knows the data owns the behaviour.** If a method needs a member's borrowed books and limit, it belongs on `Member` (`member.atLimit()`), not in a service reaching inside.

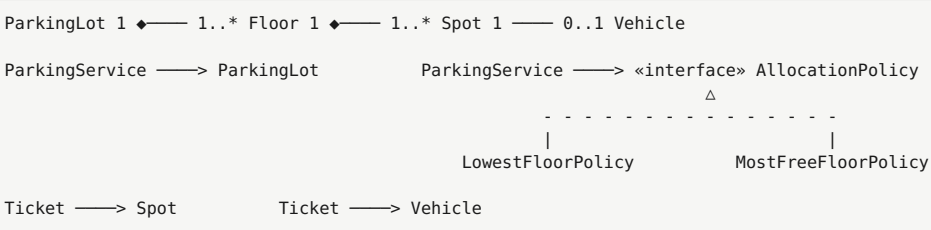
Relationships

Relationship	Means	Example	In Java
Inheritance (is-a)	a kind of the parent	`EngineeringAdmission` is an `Admission`	`extends`
Implementation	keeps a promise of an interface	`LowestFloorPolicy` is an `AllocationPolicy`	`implements`
Composition (owns)	the part can't exist without the whole	a `Floor` belongs to one `ParkingLot`	field, created inside
Aggregation (has)	the part lives on its own	a `Team` has `Employee`s	field, passed in
Association (knows)	refers to another object	a `Ticket` knows its `Spot`	field
Dependency (uses)	uses briefly, keeps no reference	a method takes a `Clock` parameter	parameter or local

Multiplicity says how many: `1` exactly one · `0..1` optional · `*` any number · `1..*` at least one. A lot has `1..*` floors; a spot has `0..1` vehicle; a member has `*` borrowed copies.

5. UML class diagrams on one page

Notation	Meaning
A box: name / fields / methods	a class. `«interface»` above the name for an interface; italics or `{abstract}` for abstract
`+` public `-` private `#` protected	visibility, written before each field or method
`- spots : List<Spot>`	field: name, then type
`+ park(type, reg) : Ticket`	method: name, parameters, then return type
solid line, hollow triangle `—▷`	inheritance (points at the parent)
dashed line, hollow triangle `- -▷`	implements an interface
filled diamond `◆—`	composition (diamond on the owner's side)
hollow diamond `◇—`	aggregation
solid line or arrow `—>`	association (knows)
dashed arrow `- ->`	dependency (uses)



In an interview, nobody grades perfect UML. Draw boxes with the key fields and methods, label each line ("has", "is a", "uses"), write the multiplicity, and talk while you draw. Five clear boxes beat fifteen cramped ones.

6. Layering: driver, service, repository, entity

Even in a single file, keep four layers apart. Each layer talks only to the one below it.

Layer	Job	In our solutions	In Spring
Driver	reads commands, calls the service, prints results and errors	`Main`	`@RestController`
Service	the rules and the order of steps; the front door (Facade)	`ParkingService`, `LibraryService`	`@Service`
Repository	storing and finding entities	in-memory `Map`s	`JpaRepository`
Entity	the things, with the behaviour they own	`Spot`, `Member`, `Book`	`@Entity`

```
interface BookRepository {                                  // the storage contract
    Optional<Book> findByIsbn(String isbn);
    void save(Book book);
}

class InMemoryBookRepository implements BookRepository {    // good enough for an interview
    private final Map<String, Book> byIsbn = new HashMap<>();
    public Optional<Book> findByIsbn(String isbn) { return Optional.ofNullable(byIsbn.get(isbn)); }
    public void save(Book book)                   { byIsbn.put(book.isbn(), book); }
}

class LibraryService {                                      // knows the interface, not the HashMap
    private final BookRepository books;
    LibraryService(BookRepository books) { this.books = books; }
}
```

Rules for the layers. Printing happens only in the driver; the service **throws** and the driver prints the message. The service never parses text. The entities never print. Swapping the in-memory repository for a database touches one class, and that's the sentence to say in an interview.

In the PDF solutions, small problems keep the maps directly inside the service to save space. The heavy problems and your own interview answers should name the repository layer.

7. Concurrency toolbox

When do you need it? Only when **shared, changeable state** can be touched by two threads at once. Read-only data and immutable objects need nothing.

The bug behind almost every concurrency question: check-then-act.

```
if (spot.isFree()) {        // thread A checks: free
                            // thread B checks: free
    spot.occupy(car);       // A parks... and B parks in the same spot
}
```

The check and the action must happen as **one step**. A thread-safe collection makes each single call safe, not two calls in a row.

Tool	Use it when	Example
`synchronized` block	one JVM; a few lines of check-then-act	find a free spot and occupy it
`ReentrantLock` with `tryLock(timeout)`	you must not wait forever, or need fairness	seat locking with a 2-second limit
`AtomicInteger`, `compareAndSet`	a single counter or value	ticket numbers, request counts
`ConcurrentHashMap` with `computeIfAbsent` / `putIfAbsent`	a shared map where "check, then insert" must be one step	a cache; "is this car already parked?"
`CopyOnWriteArrayList`	a list read often and changed rarely	listeners
Immutability (`final` fields, records)	whenever you can; nothing to lock	`Ticket`, `Product`
Optimistic locking (a **version** column)	**several servers**; conflicts are rare	update stock only if the version is unchanged
DB unique constraint or row lock; Redis `SETNX`	several servers; the database or Redis must decide	one active ticket per number plate

Optimistic locking in one line: `UPDATE item SET qty = qty - 1, version = version + 1 WHERE id = ? AND version = ?`. If zero rows changed, someone else got there first: re-read and retry. `synchronized` protects one JVM only; with three servers behind a load balancer, the database must decide.

Deadlock and lock ordering

A deadlock happens when two threads each hold one lock and wait for the other's. Neither can move, forever.

```
// Thread 1: transfer(a, b)              // Thread 2: transfer(b, a)
synchronized (a) {                       synchronized (b) {
    synchronized (b) { ... }                 synchronized (a) { ... }     // each waits for the other
}                                        }
```

The fix: always take locks in the same order, for example by id. Then both threads try to take the same lock first, and one simply waits its turn.

```
Account first  = a.id() < b.id() ? a : b;
Account second = (first == a) ? b : a;
synchronized (first) {
    synchronized (second) { /* move the money */ }
}
```

Three habits that prevent most concurrency bugs. **1.** Keep the locked part small: check and change, nothing else. **2.** Announce to listeners **outside** the lock, so a slow listener can't block everyone. **3.** If you need two locks, take them in a fixed order, or use `tryLock` with a timeout and back off.

8. Failure rules

Rule	Why
A failure must look like a failure: throw, don't return a sentence or `null`.	The caller can't mistake an error for a result. The driver prints `e.getMessage()`.
Required steps first, announcements last.	Never send "booked!" for a booking that then fails.
Check before you change.	Work out whether you *can* (change available, seat free) before touching any state.
Halfway failure: clean up in reverse, then rethrow.	Last done, first undone: refund, then release, then report.
Catch only what you expect.	`catch (Exception e)` hides real bugs under a friendly message.
What can't be undone gets compensated.	A refund, a "cancelled" message. Database rollback can't un-send an SMS.
One failing listener must not stop the others.	A try/catch around each listener call: "carry on" is right for side effects.
Missing but normal → Null Object. Missing and wrong → throw.	No coupon: do nothing. No payment response: stop.
"X hasn't happened in N minutes" is a scheduled job.	Nothing happens, so there's no event to listen to. Only a clock notices.

9. Pattern cheat sheet

What the requirement sounds like	Pattern
"a different formula or algorithm per type", "the rules change every year"	**Strategy**
"the type arrives as text and we must pick the right object"	**Factory**
"must be the same everywhere", "one source of truth"	**one shared instance**, created once and injected
"2 required, many optional, a rule across fields, never changes after"	**Builder**
"when X happens, also do A, B, C" and "X still counts if one fails"	**Observer**
"allowed in some statuses, refused in others", many statuses × many operations	**State**
"extras that stack, in any combination"	**Decorator**
"their SDK doesn't match our interface, and we can't edit it"	**Adapter**
"undo / redo", "do it later", "retry", "a log of actions"	**Command**
"up to X this one decides, up to Y that one, above that someone else"	**Chain of Responsibility**
"the same steps in the same order; only one or two steps differ"	**Template Method**
"several steps in order, and several callers need it"	**Facade**
"groups inside groups", "the total of any unit"	**Composite**
"only X may…" · "slow, and asked for often" · "load only when needed"	**Proxy**: guard · remember · delay
"thousands of objects with the same heavy data"	**Flyweight**
"go through all items without knowing how they're stored", "results arrive page by page"	**Iterator**
"save and restore the whole state", "snapshot", "restore from backup"	**Memento**
"missing is normal, and then nothing should happen"	**Null Object**
"copy an existing, set-up object and change a little", "duplicate", "order again"	**Prototype**
"many objects must coordinate; one hub decides who reacts to what"	**Mediator**
"a fixed set of kinds, and new operations or reports keep coming"	**Visitor**
"no A, no X" (payment before the order)	**nothing**: a required step, called directly
"X hasn't happened in N minutes"	**nothing**: a scheduled job
"kinds that differ only in numbers"	**nothing**: an enum with fields
"always the same few fields"	**nothing**: a record
"items never contain items" · "a dozen items, a megabyte each" · "one call to one class"	**nothing**: a list, just load it, call it directly
"maybe next year"	**nothing**: YAGNI

Look-alikes

Pair	The question that separates them
Decorator vs Proxy	Adding, or controlling?
Decorator vs Chain	Every link runs and adds, or a link can stop it?
Strategy vs Template Method	The whole calculation differs, or a few steps inside a fixed order?
Strategy vs State	Picked by the caller, or changes itself as things happen?
Observer vs Mediator vs Facade	Announce and forget · a hub with the rules · one front door for outside callers
Flyweight vs caching Proxy	Saving memory, or saving time?
Builder vs Prototype	Build from nothing, or copy something finished?
Memento vs Command undo	Snapshot the whole state, or reverse one action?
Composite vs a plain list	Do items contain items, to any depth?
Visitor vs adding a method	Are the kinds fixed while operations keep coming?
Null Object vs throw	Is "missing" normal, or a failure?
Observer vs scheduled job	Did something happen, or did something *not* happen?

10. The machine-coding format

Every Part B is written the way Flipkart-, Swiggy- and Razorpay-style machine-coding rounds are run. You get a list of **commands**, one per line, and the **exact output** they must produce. How you design the classes is entirely up to you. Only the input and output are fixed.

```
input.txt                       expected.txt
---------                       ------------
CREATE_LOT 2 2 2 1              Created lot: 2 floors, each with 2 car, 2 bike, 1 truck spots
PARK CAR MH12AA1111 09:00       Ticket T1: CAR MH12AA1111 at F1-C1, in 09:00
PARK BUS MH12BU0000 10:15       Unknown vehicle type BUS
```

Running and checking

Where	How
Terminal (Mac, Linux)	`javac Main.java` → `java Main < input.txt > output.txt` → `diff expected.txt output.txt`. No output from `diff` means a perfect match.
Windows	The same `javac` and `java` lines, then `fc expected.txt output.txt`
IntelliJ	Run → Edit Configurations → Modify options → **Redirect input from** → choose `input.txt`. Then paste the console output into a file and compare.

The driver skeleton every solution uses

```
public class Main {
    public static void main(String[] args) {
        Service service = new Service();                  // build the objects once
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");              // words separated by spaces
            try {
                switch (p[0]) {
                    case "PARK" -> { /* call the service, then print the result line */ }
                    default     -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (DomainException e) {                 // your own exception type
                System.out.println(e.getMessage());       // print the error and carry on
            }
        }
    }
}
```

The gaps that cost you the most marks (these come straight from your mistake tracker).
Exact text. One missing space or a capital letter is a mismatch. Paste the expected line as a comment above your print statement and build the string to match it.
One public type per file. In a single `Main.java`, only `Main` is `public`.
Operator order. `"a" + b + flag ? x : y` doesn't compile. Bracket the choice: `+ (flag ? x : y)`.
Errors throw. The service throws, and the driver prints `e.getMessage()`.
Run early and often. Compile after every class, not once at the end.

11. Interview behaviour

Moment	What to say or do
Starting	"Let me make sure I understand. The core operations are X, Y and Z, and I'll leave out W unless you want it."
Thinking	Think aloud: "I'm considering A or B. B is simpler, but A handles new types without edits. Since you said types change often, I'll take A."
Choosing a pattern	Name the requirement first, then the pattern: "Fees differ per vehicle and change yearly, so a Strategy."
"Isn't that over-engineered?"	Concede first: "You'd be right if… For this requirement, though, … Where nothing was needed I used nothing."
"I don't know"	Don't bluff. "I haven't used that. Here's how I'd reason about it…, and I'd confirm by checking…"
"That's wrong"	Re-check honestly. If they're right: "You're right, because… I'd change it to…" If not: explain calmly, with the reason.
Running late	Say it, and choose: "I'm short on time, so I'll code the core booking path and describe the rest."

12. Master tracker

Fill in the dates by hand. "Score" is your total from the 10-point checklist in each solution. A score below 7 means: do the day-10 re-do as a full attempt, not just Part A.

PDF	Problem	Size	Attempted	Day 3	Day 10	Day 30	Score
1	Tic-Tac-Toe	small					
1	Vending Machine	core					
1	Library Management	core					
1	Parking Lot	heavy					
2	Deck of Cards	small					
2	ATM	core					
2	Splitwise	core					
2	Elevator System	heavy					
3	LRU Cache	small					
3	Rate Limiter	core					
3	Digital Wallet	core					
3	BookMyShow	heavy					
4	Snake and Ladder	small					
4	Cache with TTL and eviction	core					
4	Meeting Room Scheduler	core					
4	Chess	heavy					
5	Logging Framework	small					
5	Hotel Booking	core					
5	Task Scheduler	core					
5	Food Delivery	heavy					
6	Traffic Signal / Amazon Locker	small					
6	Notification System	core					
6	Inventory Management	core					
6	Ride-sharing	heavy					

One last thing. These PDFs give you the problems, the hints and the answers. What makes the difference is the order: attempt before you look, say your reasoning aloud, and come back to each problem three times. Good luck.